

Min-Max Local/Global Alternation (Formulation A)

Config selector: ppo.method = constrained

Source: `crl/ppo_continual.py` (PPOAlternationTrainer._constrained_task,
_train_first_task, run) + `crl/ppo/trainer.py`
(LocalTrainer.train, GlobalTrainer.train)

Math: docs/Objective_for_Continual_Reinforcement_Learning.pdf, eqs 5-32.

Idea

Tasks 1..K arrive in sequence. A single shared-trunk multi-head GLOBAL policy

$\pi_G = \pi_\phi$ must perform on every seen task. For task $k \geq 2$ each cycle

runs two phases:

(1) LOCAL phase: from $\theta^0 = \phi$ (eq 2) train a throwaway copy with
standard PPO on task k ONLY (eqs 7-8). Freeze it; its rollout value
 V_k^L (eq 19) is the current-task reference / floor.

(2) GLOBAL phase: update ϕ to maximize the past-task lead

$\sum \omega_i (V_i^{\pi_G} - V_i^{\pi_L})$ subject to the current-task
floor $V_k^G \geq V_k^L$, enforced by a one-sided squared shortfall and a
single dual multiplier μ (eqs 10-12). Saddle point: max over ϕ ,
min over $\mu \geq 0$.

Only the ACTOR is constrained; the critic, GAE, value loss and entropy are
standard PPO (see doc 03). Rollouts are replay-free: every task is re-collected
fresh in its own environment each iteration.

Inputs

family : ordered list of K tasks (Atari games), each with a threshold
global_policy : shared-trunk, per-task actor+critic heads (task_conditioned=F)
cfg.cycles_per_task : local+global cycles per task
ppo.task1_iters, ppo.local_iters, ppo.global_iters
ppo.clip_ratio, lr, vf_coef, ent_coef, gae_lambda, n_envs, n_steps
dual controller (mu) config: init value, step eta_mu, max_value, threshold eps
seed

Outputs

global_policy after all K tasks (final_policy.pt)
eval_matrix : lower-triangular raw-score forgetting matrix (eval_matrix.json)
per-phase logs (probe / global_diag / gaps rows)

Top level (PPOAlternationTrainer.run -> method == "constrained")

TRAIN FIRST TASK (_train_first_task):
standard PPO trains global_policy on task 1 for task1_iters

```

    (no past tasks, no constraint; LocalTrainer.train on the GLOBAL net itself)
    evaluate row 1 of eval_matrix (raw greedy+stochastic pooled score); log

for k = 2 .. K:
    _constrained_task(k)                # min-max consolidation
    evaluate row k of eval_matrix; append; log
save eval_matrix, final_policy

```

_constrained_task(k)

```

task_k      <- family.tasks[k-1]
past_tasks <- family.tasks[0 .. k-2]
for cycle = 0 .. cfg.cycles_per_task - 1:

    # ----- LOCAL PHASE (eqs 7-8, 16-21) -----
    local_policy <- clone_policy(global_policy, trainable=True) #  $\theta^0 = \phi$ 
    LocalTrainer.train(local_policy, task_k, num_iters=local_iters,
                        seed = seed + 1000*k + 13*cycle)         # standard PPO
    frozen_local <- clone_policy(local_policy, trainable=False) # freeze  $\pi_L$ 
    ref_current  <- V_k^L = eval_value(frozen_local, task_k)    # eq 19,
                        # on-policy STOCHASTIC discounted return, constraint_episodes

    # ----- GLOBAL PHASE (eqs 10-12, 22-32) -----
    mu_ctrl.reset()                # fresh dual per global phase
    omega <- [1/k] * (k-1)         # uniform weights, past tasks
    GlobalTrainer.train(global_policy, task_k, past_tasks,
                        ref_current = V_k^L, mu_ctrl, omega,
                        eps, num_iters = global_iters,
                        local_policy = frozen_local) # for KL/BC diagnostics
    log {"phase":"gaps", task=k, cycle, V_k_ref_local = ref_current}

# (optional experiment global_probe_head_only: copy local weights into global,
# freeze the trunk, move only per-task heads during the global phase; then
# restore full trainability. Off by default.)

```

LocalTrainer.train (standard PPO, the local player; eqs 16-21)

```

optimizer <- Adam(trainable params, lr, eps=1e-5)
collector <- RolloutCollector(task, n_envs, n_steps, seed)
met <- 0
for it = 0 .. num_iters-1:
    batch <- collector.collect(policy, gae_lambda) # fresh rollout + GAE
    optimize_batches(policy, opt, streams=[batch], actor_coeffs=[1.0]) # doc 03
    probe(...) # periodic retention log
    gscore <- greedy score every stop_eval_every iters (>= min_iters), else None
    if gscore >= threshold: met += 1 else: met <- 0

```

```

if met >= patience: EARLY STOP
# stopping: num_iters exhausted OR greedy threshold met `patience` times

```

GlobalTrainer.train (constrained global player; eqs 10-12, 22-32)

Initialized quantities:

```

optimizer      <- Adam(trainable params of global_policy)
past_collectors[i] <- RolloutCollector(past_task_i, seed+101+i)  # replay-free
cur_collector   <- RolloutCollector(task_k,      seed+7)
mu <- mu_ctrl.value ; shortfall <- 0 ; constraint <- 0 ; V_k^G <- NaN

```

for it = 0 .. num_iters-1:

```

# ---- fresh rollouts (all tasks re-collected every iteration) ----
if past_task_sampling == "sample":
    pick one random past task j; past_batches=[collect_j];
    past_coeffs=[omega[j] * n_past]          # unbiased estimate of full sum
else:
    past_batches <- [collect(global, past_task_i) for all i]
    past_coeffs <- omega[:len(past_tasks)]    # = 1/k each
cur_batch <- cur_collector.collect(global, gae_lambda)

# ---- refresh constraint value V_k^G and mu every constraint_every iters ----
if it % constraint_every == 0:
    V_k^G <- fresh estimate of the global's current-task value      # eq 27
                    (critic on cached no-op start states when constraint_use_critic,
                     else a Monte-Carlo rollout estimate)
    # one-sided shortfall of the CURRENT task (eq 9 hinge):
    #   "ratio" form : sf = max(0, V_L - V_G) / max(|V_L|, tau)
    #   "floored"    : sf = max(0, (V_L - V_G) - delta*max(|V_L|, tau))
    sf_start <- _sf(V_k^L, V_k^G)
    if intermediate states available (Run B):      # 50 no-op + 50 mid states
        per-state shortfalls averaged; blend
        shortfall <- 0.5*sf_start + 0.5*mean(sf_interim)
        constraint <- 0.5*sf_start^2 + 0.5*mean(sf_interim^2)      # F_G, eq 9/33
    else:
        shortfall <- sf_start
        constraint <- sf_start^2                                # F_G = (.)^2
    mu <- mu_ctrl.update(constraint, dual_eps)    # eqs 31-32:
        #   mu <- max(0, mu + eta_mu*(F_G - eps)), clipped to max_value
        #   (dual_eps = 0 for "floored"; = eps otherwise)

coeff_k <- mu * 2.0 * shortfall                  # differentiated hinge, eq 30/32

# ---- one PPO update over ALL streams (doc 03) ----
streams <- past_batches + [cur_batch]

```

```

coeffs <- past_coeffs + [coeff_k]          # per eq 29:
      # past task i actor weight = omega_i      (maximize lead)
      # current task k actor weight = mu*2*shortfall (enforce floor)
optimize_batches(global, opt, streams, coeffs, bc=optional_KL)
      # inside: coeffs are RENORMALIZED by their sum (PPO-Lagrangian stability),
      # clipped surrogate per stream, standard value loss + entropy, grad clip.

probe(...) ; optional past_monitor(...) ; optional diagnostics(...)
gscore <- greedy score of global on task_k (early-stop check)
if gscore >= threshold: met += 1 else met <- 0
if met >= patience: EARLY STOP

# stopping: global_iters exhausted OR global reaches task_k greedy threshold
# `patience` times (past tasks are held up by the omega lead term).

```

What is updated, and when

```

phi (global params) : every iteration, by optimize_batches (ascends eq 29).
mu (dual)             : every constraint_every iters (eqs 31-32); reset each
                      : global phase, warm-started from config.
V_k^L                 : computed ONCE per cycle after the local phase (frozen).
V_k^G                 : refreshed every constraint_every iters (drives shortfall).

```

Key intermediate quantities

```

V_k^L = ref_current   : frozen local value = current-task floor (eq 7/19)
V_k^G                 : global's current-task value (eq 27) -- moves
F_G = constraint      : one-sided squared shortfall (eq 9); constraint metric
shortfall             : max(0, V_L - V_G) hinge (relative or floored)
coeff_k = 2*mu*sf     : current-task actor gradient weight (eq 30)
omega_i = 1/k         : past-task actor gradient weight

```

Implicit assumptions

- "Access to all previously seen environments" -> fresh old-task rollouts each iter (replay-free, eq 4 note), never a replay buffer.
- Only the actor carries the constraint; the critic head is trained from scratch by PPO's value loss on the shared trunk (not seeded).
- Uniform weights $\omega_i = 1/k$ throughout.
- Reported eval = raw game SCORE; the constraint uses discounted value V. (memory: report performance not value; greedy-only reporting is a separate knob -- here the report pools greedy+stochastic.)

Code <-> equation map

```

clone theta^0 = phi ..... eq 2

```

local objective $\max V_k^{\{\pi_L\}}$	eqs 7-8, 16-21
weighted past objective $\sum \omega_i V_i$	eq 5
improvement split (current vs past)	eq 6
one-sided squared shortfall F_G	eq 9 / eq 33
constrained global problem + Lagrangian	eqs 10-11
saddle point $\max_{\phi} \min_{\mu}$	eq 12
per-task policy gradient (global)	eqs 22, 27-28
global primal update (actor weights)	eq 29
dual gradient / μ ascent / projection	eqs 30-32